COMPILER DESIGN

UNIT IV

Content

Compiler:

Overview of Compiler-environment
phases of compiler
phase and pass
lexical analyzer
Bootstrapping
LEX

Top Down Parsing:

Top down parsing technique
Recursive decent parsing with back tracking
Ambiguous grammar
Elimination of left recursion
Left factoring
Predictive parsing
LL(1)

Language Translator

- A **Language translator** is a program which accepts inputs as a program written in source language (high- level language) and produces an output, a program in another language i.e., target language
- **Basic functions of language translator**:

✓ Translating the high level language program into m/c language program

✓ Detection and reporting of errors.

- **Types of Language Translator:**

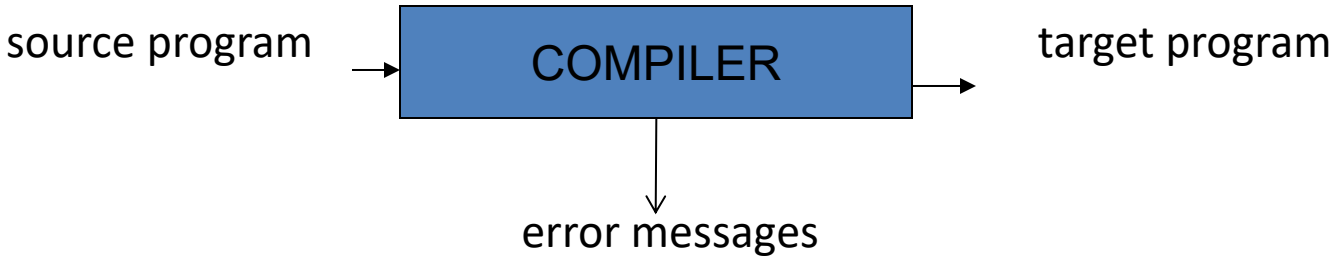
➢ **Compilers**

➢ **Interpreters**

➢ **Assemblers**

Language Translators

- **Compiler**: Is a program takes a program written in a source language and translates it into an equivalent program in a target language.



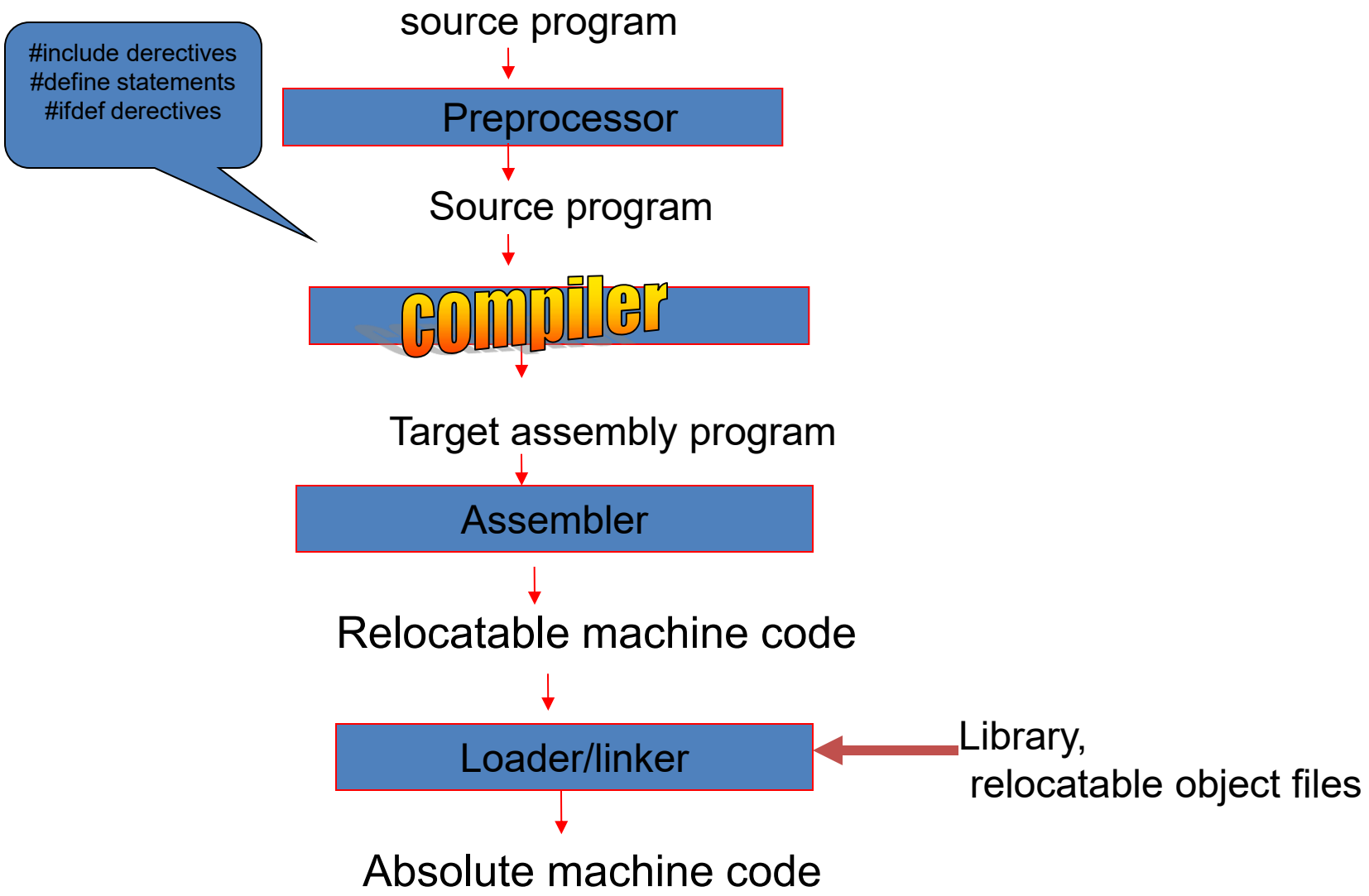
- **Interpreter**: An interpreter is a program which translates a high level language program into machine language and interprets each instruction and executes it immediately.

- **Assembler**: An Assembler is a program which translates assembly language to machine level language.

Differences between compiler and interpreter

Compilers	Interpreter
1.Compiler checks errors at compile time	1.Interpreter checks for errors even at runtime
2. A compiled program takes less time for execution	2.Takes more time
3. Compilers requires more memory while translation	3. Requires less memory space
4. Some compiled programming languages are C , C++ etc	4. Interpreter languages are LISP ,APL, Smalltalk, JavaScript, PHP, Python, Perl, Ruby etc

Where the compiler fits



Logical phases of compiler

The compiler implementation process is divided into two parts:

Analysis of a source program

Synthesis of a target program

Analysis involves analyzing the different constructs of the program

Analysis consists of three phases:

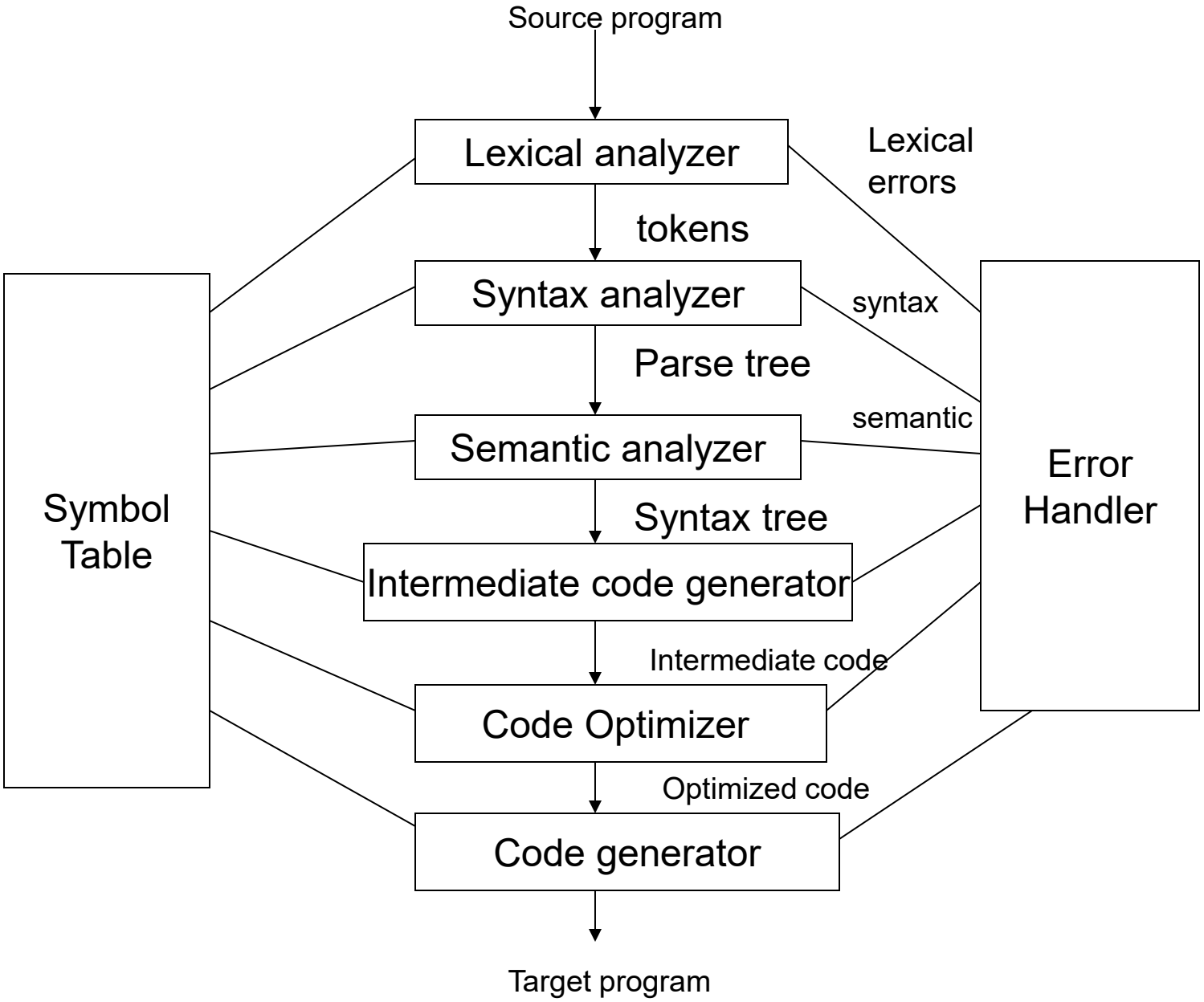
- **Lexical (Linear) analysis or scanner**
- **Syntax Analysis or parser**
- **Semantic analysis**

Synthesis of a target program includes three phases:

- **Intermediate code generator**
- **code optimizer**
- **code generator**

- Each phase transforms the source program from one representation into another representation.
- They communicate with error handlers.
- They communicate with the symbol table.

Phases of compiler



Lexical Analyzer

Lexical Analyzer reads the source program character by character to produce tokens.

Ex:

position= initial + rate* 60 (C statement)

position, initial, rate : identifiers

= : assignment symbol

+, * : operators

60 : number (constant)

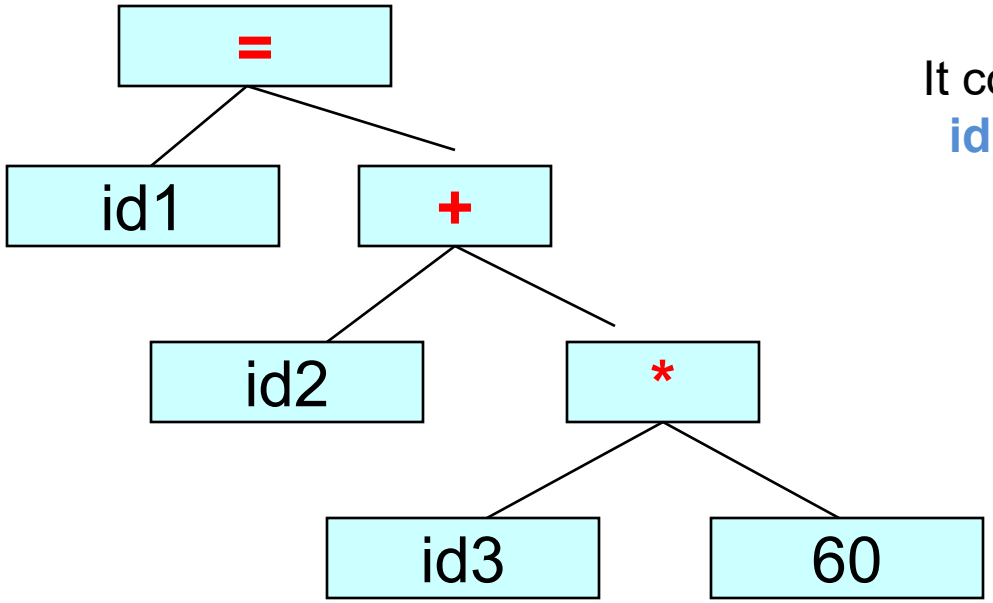
After lexical analysis phase the statement becomes :

id1 = id2 + id3 * 60

All the identifiers are stored in symbol table

Syntax Analyzer

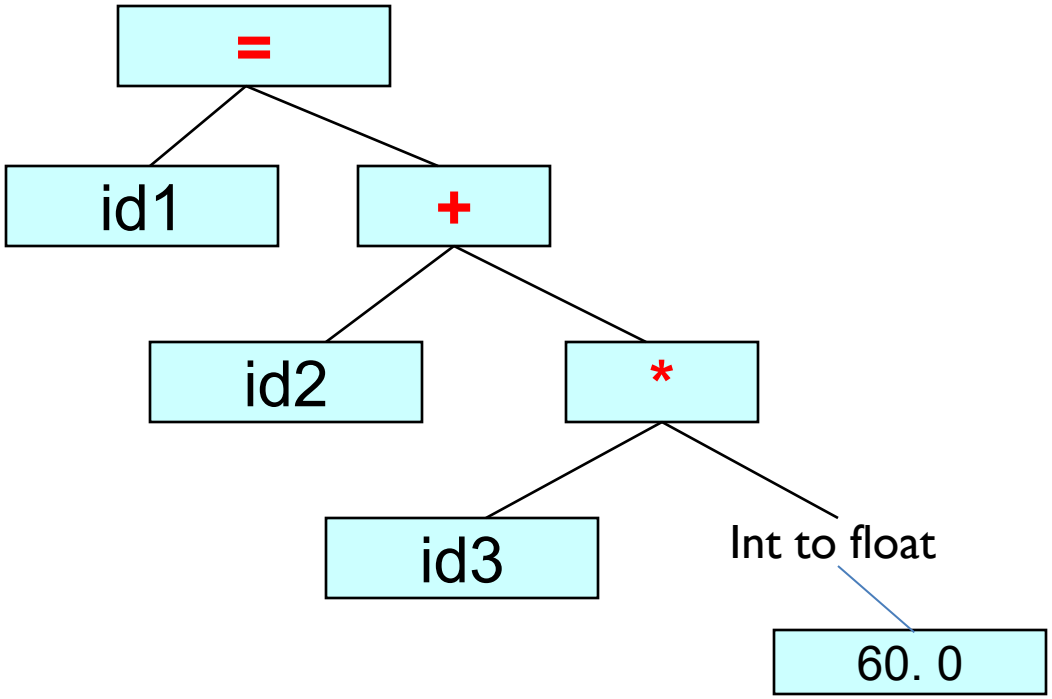
- A **Syntax Analyzer** creates the syntactic structure (generally a parse tree) of the given program.
- This phase receives the outcome of the lexical phase as input.



It constructs parse tree for the statement
id1 = id2+id3 * 60

Semantic Analyzer

- A semantic analyzer checks the source program for semantic errors and collects the type information for the code generation.
- Type-checking and type conversion are two important parts of semantic analyzer.



Intermediate Code Generation

- A compiler may produce an explicit intermediate codes representing the source program.
- One such intermediate representation is :

Three address code

Ex: id1 = id2 + id3 * (int to float)60

temp1=inttofloat(60)

temp2=id3 *temp1

temp3=id2+temp2

id1=temp3

Code Optimizer
(for Intermediate Code Generator)

- The code optimizer optimizes the code produced by the intermediate code generator in the terms of time and space.

temp1:=id3*60.0

id1:=id2+temp1

temp1=inttofloat(60)
temp2=id3 *temp1
temp3=id2+temp2
id1=temp3

Code Generator

- Produces the target language in a specific architecture.

(assume that we have an architecture with instructions whose at least one of its operands is a machine register)

```
MOVF    id3, R2
MULF    #60.0,R2
MOVF    id2 ,R1
ADDF    R2,R1
MOVF    R1,id1
```

temp1:=id3*60.0
id1:=id2+temp1

Symbol Table Management :

When an identifier is recognized by the lexical analyzer , the identifier is stored in symbol table.
The symbol table can also store the type and value of each identifier.

Error Handler :

It is a system program of a compiler used for detecting and reporting errors, encounter in each phase of a compiler.

Phases and Passes

Pass	Phase
1.Pass is a physical scan over a source program The portions of one or more phases are combined into a module called pass	1. A phase is a logically cohesive operation that takes i/p in one form and produces o/p in another form
2. Requires an intermediate file between two passes	2. No need of any intermediate files in between phases
3. Splitting into more no. of passes reduces memory	3. Splitting into more no. of phases reduces the complexity of the program
4. Single pass compiler is faster than two pass	4. Reduction in no. of phases, increases the execution speed

Lexical Analyzer

The lexical analyzer is the first phase of compiler

- **Basic functions of lexical analyzer:**

1. It reads the characters and produces as output a sequence of tokens
2. It removes comments, white spaces (blank ,tab ,new line character) from the source program.
3. If lexical analyzer identifies any token of type identifier ,they are placed in symbol table
4. Lexical analyzer may keep track of the no. of new line characters seen, so that a line number can be associated with an error message.

- **Terms used in lexical analysis**

Lexeme: Lexemes are the smallest logical units of a program. It is sequence of characters in the source program for which a token is produced.

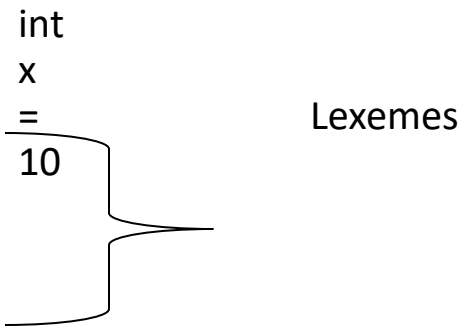
Tokens: Class of similar lexemes are identified by the same token.

Pattern : Pattern is a rule which describes a token

Ex: Pattern of an identifier is it consists of letters and digits but the first character should be a letter.

int x = 10;

int is a lexeme for the token **keyword**
x is a lexeme for the token **identifier**



The lexical analyzer must also pass additional information along with token, when a pattern matches more than one lexeme. These items information are called attributes for tokens.

Generally a token of type identifier has single attribute, i.e. pointer to the symbol table entry.

Ex: x = y + 2

Lexeme	< token , Token attribute >
x	<identifier , pointer to the symbol table entry>
=	<assign-op , >
y	<identifier , pointer to the symbol table>
+	<Add-op , >
2	<const , integer value 2>

Regular Expressions & Transition Diagrams

A notation which is used to specify the token is called **Regular Expression .** Regular expressions are constructed for any token over an alphabet.

Ex: A Pascal identifier is defined as *"letter followed by zero or more letters or digits"*

The regular Expression that denotes an identifier by using above pattern is

id=L(L+D)*

Where L={ a,b…z}

D ={ 0,1,..9}

The Lexical analyzer phase recognizes the tokens specified in Regular Expression using "Transition Diagram".

Application of Finite Automata to Lexical Analyzer

In Lexical analyzer the finite automata is used to recognize the tokens.

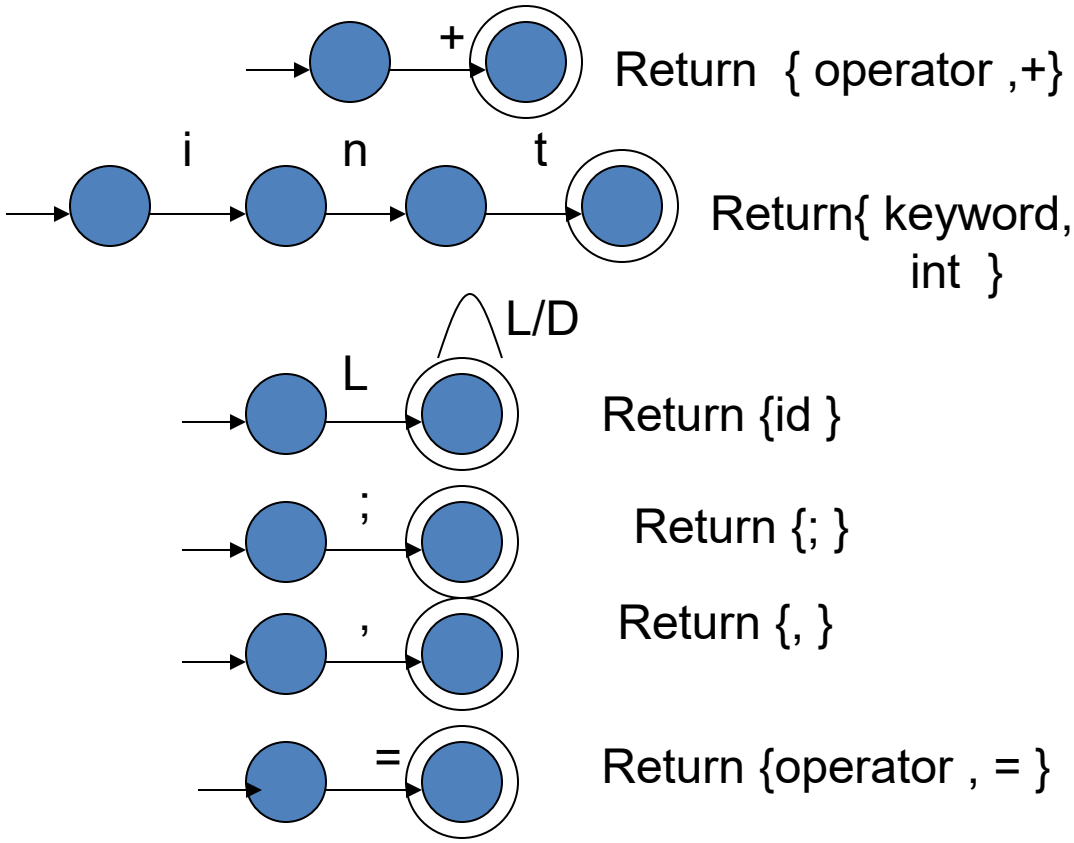
Ex:

…….

……..

```
 int x, y , z;
 z=x+y;
```

……..



•**As characters are read, the relevant TDs are used to attempt to match lexeme to a pattern.**

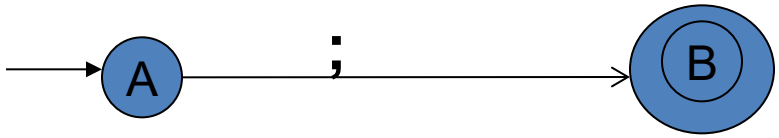
Ex: **sum=a+b;**

To recognize the tokens in above statement, three different transition diagrams are used.

1) TD to recognize identifiers (sum , a, b)

2) TD to recognize operators (= , +)

3) TD to recognize special symbols (;)



Above TD is used to recognize ; symbol

Write a Lexical Analyzer program to identify keywords, identifiers, operators, Special characters.

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
FILE *fp;
static char ch;
char keyword[8][10]={“int”,”
main”,”printf”,”float”,”if”,”else”,”break”,”char”};
char brace[10]={‘(‘,’)’,’{‘,’}’,’[‘,’]’};
char oper[10]={‘+’,’-‘,’*’,’/’,’%’,’<’,’>’,’=’};
char sym[10]={‘.’,’,’,’;’,’:’,’|’,’#’,’””};
char identifier[32];
char number[10];
int i;
```

```
void scantoken();
void main()
{
  fp=fopen(“input.c”,”r”);
  ch = getc(fp);
  printf(“\ntoken----------lexeme\n”);
  while(!feof(fp))
    {
          scantoken();
     }
  fclose(fp);
}
```

```
void scantoken()
{
int j;
for(i=0;i<10;i++)
{
        if(brace[i]==ch)
        {
                printf("Brace----
%c\n",ch);
                ch=getc(fp);
        }
}
for(i=0;i<10;i++)
{
        if(oper[i]==ch)
        {
                printf("operator----
%c\n",ch);
                ch=getc(fp);
        }
}
for(i=0;i<10;i++)
{
        if(sym[i]==ch)
        {
        printf("Symbol----
%c\n",ch);
                ch=getc(fp);
        }
}
```

```
if(isalpha(ch))
{
        identifier[0]=ch;
        identifier[1]='\0';
        j=1;
        ch=getc(fp);
        while(isalnum(ch))
        {
                identifier[j++]=ch;
                ch=getc(fp);
        }
        identifier[j]='\0';
        if(key()==1)
        {
                printf("keyword------
%s\n",identifier);
        }
        else
        {
                printf("identifier-----
%s\n",identifier);
        }
}
```

```
elseif(isdigit(ch))
{
        number[0]=ch;
j=1;
ch=getc(fp);
while(isdigit(ch))
{
        number[j++]=ch;
        ch=getc(fp);
}
        identifier[j]='\0';
printf("number----%s\n",number);
}
elseif(isspace(ch))
{
        ch=getc(fp);
}
}
```

```
int key()
{
        int flag=0;
        for(i=0;i<8;i++)
{
        if(strcmp(keyword[i],identifier)
==0)
        {
                flag=1;
                break;
        }
}
return flag;
}
```

Introduction to LEX

1. **LEX** stands for Lexical Analyzer.

2. LEX is a UNIX utility which generates the lexical analyzer.

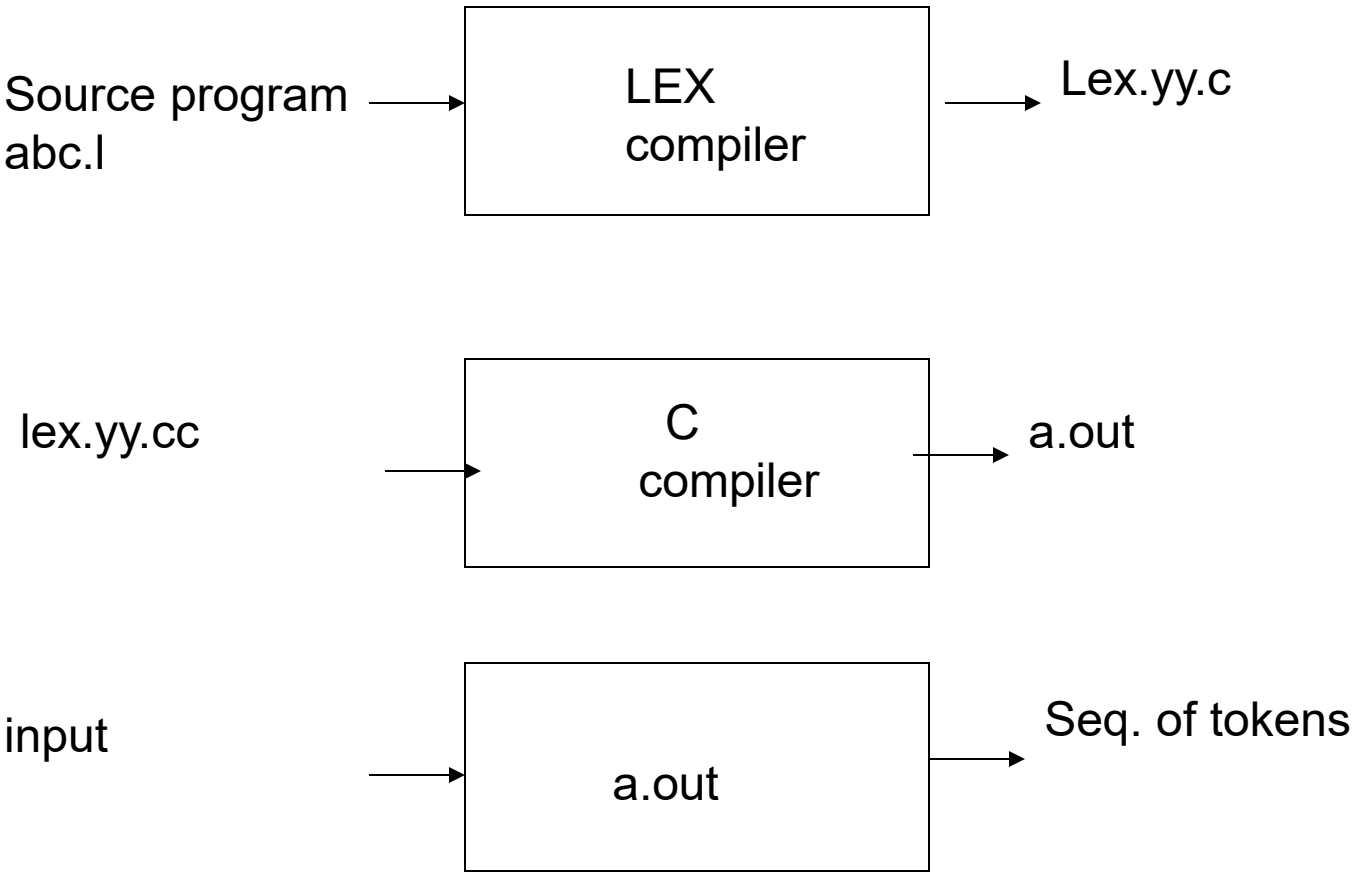
3. When Lex receives input in the form of a file or text, it attempts to match the text with the regular expression.

4. It takes input one character at a time and continues until a pattern is matched.

5. If a pattern can be matched, then Lex performs the associated action (which may include returning a token)

6. A lex file *(files in Lex have the .l extension eg: first.l) is passed through the* lex utility, and produces output files in C (lex.yy.c).

Creating a lexical analyzer with LEX



LEX Program Format

LEX is a tool which is use to specify lexical analyzer
The LEX program consists of three parts:

```
%{
 Declarations
   %}
%%
Translation rules
%%
auxiliary procedures
```

- The declaration section includes declarations of variables, constants and regular definitions
- The translation rules are of the form

  p1 {action1}
  p2 {action2}

  Where p1 ,p2 are regular expressions and each action is describes what action should take when a pattern matches a lexeme.

The third part holds function or procedures needed by the actions

Lex program to identify Operators,Identifiers:

```
%{
    #include<stdio.h>
%}
%%
“+” {printf(“plus operator”);}
“-“ {printf(“minus operator”);}
“*” {printf(“multiplication operator”);}
“/” {printf(“division operator”);}
<  |
>  |
<=  |
>=  { printf(“relational operator”);}
?: {printf(“conditional operator”);}
[a-z A-Z][a-z A-Z]*[0-9]* {printf(“identifier”);}
%%
main()
{
yylex();
}
int yywrap()
{
return 1;
}
```

Built-in Functions of LEX

Function	Meaning
yylex()	The function that starts the analysis. It is automatically generated by Lex.
yywrap()	This function is called when end of file (or input) is encountered. If yywrap() returns 0, the scanner continues scanning, while if it returns 1 the scanner returns a zero token to report the end f file.
yyless(int n)	This function can be used to push back all but first „n“ characters of the read token.
yymore()	This function tells the lexer to append the next token to the current token.
yyerror()	This function is used for displaying any error message.

Built-in Variables of LEX

Variables	Meaning
yyin	Of the type **FILE***. This point to the current file being parsed by the lexer. It is standard input file that stores input source program.
yyout	Of the type FILE*. This point to the location where the output of the lexer will be written. By default, both yyin and yyout point to standard input and output.
yytext	The text of the matched pattern is stored in this variable (char*) i.e. When lexer matches or recognizes the token from input token the lexeme stored in null terminated string called yytext. OR This is global variable which stores current token.
yyleng	Gives the length of the matched pattern. (yyleng stores the length or number of character in the input string) The value in yyleng is same as strlen() functions.
yylineno	Provides current line number information. (May or may not be supported by the lexer.)
yylval	This is a global variable used to store the value of any token.

Regular Expressions of LEX

A-Z, 0-9, a-z	Characters and numbers that form part of the pattern.
.	Matches any single character except newline character „\n”.
-	Used to denote range. Eg: A-Z implies all characters from A to Z.
[ ]	A character class. Matches any character in the brackets. If the first character is ^ then it indicates a negation pattern. Eg: [abc] matches either of a, b, and c.
*	Match zero or more occurrences of the preceding pattern. Eg: a.*z matches any string that starts with “a” and end with “z” such as “az”, “abcdz”,“afgfggfhz”
+	Matches one or more occurrences of the preceding pattern. Eg: x+ matches “x”, “xx”, “xxx” or (ab)+ matches ab,abab,ababab, and so on

?	Matches zero or one occurrences of the preceding pattern. Eg: -?[0-9]+
$	Matches end of line as the last character of the pattern.
{ }	Indicates how many times a pattern can be present. Eg: A{1,3} implies one or three occurrences of A may be present.
\	Used to escape meta characters. Also used to remove the special meaning of characters as defined in this table. Eg: “\t” for tab
^	Negation.
|	Logical OR between expressions. Eg: twelve|12 matches either “twelve ” or “12”
/	Look ahead. Matches the preceding pattern only if followed by the succeeding expression. Eg: A0/1 matches A0 only if A01 is the input.
( )	Groups a series of regular expressions. Eg: (ab|cd)?ef matches “abef” ,”cdef” or just “ef”

Simple LEX program

```
%%
[0-9]+     {  printf("%s is a NUMBER" ,yytext);}
.*         {  printf("%s is not a NUMBER" , yytext); }
%%
main()
   {
        yylex();
    }
int yywrap()
   {
         return 1;
    }
```

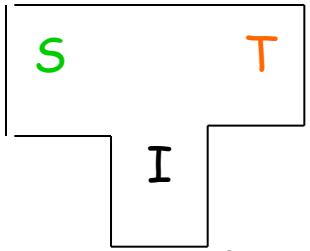
LEX Program Example

```
%{
    LT , LE ,EQ ,NE,GT ,GE ,IF ,THEN,ELSE ,ID,NUMBER ,RELOP
 %}

/* REGULAR DEFINITIONS*/
 letter          [ A – Z   a- z]
 digit           [ 0  -  9]
 id            {letter}({ letter| digit})*

%%
if            { return ( IF);}
else          { return (ELSE); }
{id}            { yvlval =install_id(); return (ID)}
"=="            { yvlval=EQ ; return(RELOP)}
%%
install_id() {      }
```

Bootstrapping

- Compiler is a complex program and should not be written in assembly language
- How to write compiler for a language in the same language (first time!)?
- First time this experiment was done for Lisp
- Initially, Lisp was used as a notation for writing functions.
- Functions were then hand translated into assembly language and executed
- McCarthy wrote a function eval[e,a] in Lisp that took a Lisp expression e as an argument
- The function was later hand translated and it became an interpreter for Lisp

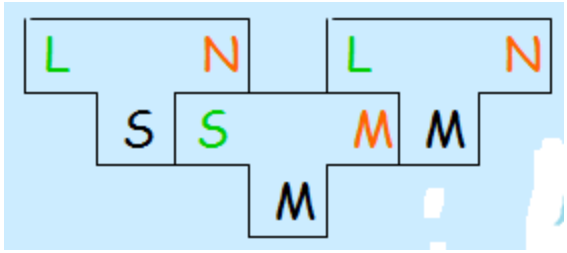
- The facility offered by a language to compile itself is the essence of Bootstrapping.
- A compiler can be characterized by three languages: the source language (S), the target language (T), and the implementation language (I)
- The three language S, I, and T can be quite different. Such a compiler is called cross-compiler
- This is represented by a T-diagram as:



- In textual form this can be represented as

$S_I T$

- Write a cross compiler for a language L in implementation language S to generate code for machine N
- Existing compiler for S runs on a different machine M and generates code for M
- When Compiler $L_S N$ is run through $S_M M$ we get compiler $L_M N$



Top down Parsing:

- Top down parsing methods
- Recursive Decent parsing with back tracking
- Elimination of left recursion
- Left factoring
- Predictive parsing
- LL(1) grammar and parsing

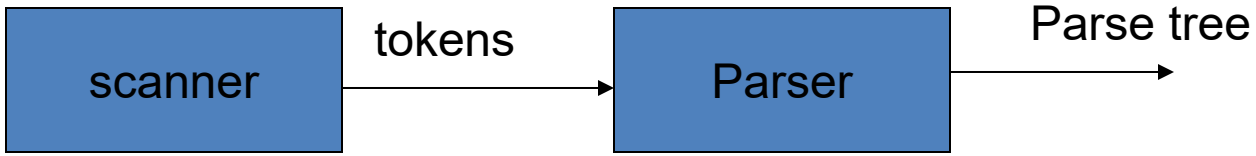
Syntax Analyzer or Parser

- Syntax analyzer involves grouping the tokens into grammatical phrases and analyze the syntax of the source program by constructing parse tree.

- Syntax analysis defines a set of production rules for describing the syntax of language known as grammar (Context Free Grammar).

Role of Parser:

- It obtains a string of tokens from the lexical phase.
- It groups the tokens by constructing ***parse tree***
- Model using CFG
- Recognize using push down automata
- It should report any syntax error in the program and also recover from the errors.



context-free grammar

- A context-free grammar (CFG) G is a quadruple (V, Σ, R, S) where
- V: a set of non-terminal symbols
- Σ: a set of terminals ($V \cap \Sigma = \emptyset$)
- R: a set of rules ($R: V \rightarrow (V \cup \Sigma)^*$)
- S: a start symbol.

Example of Context-Free Grammar

$$S \rightarrow aSb \mid \lambda$$

productions

$$P = \{S \rightarrow aSb,\ \ S \rightarrow \lambda\}$$

$$G = (V, T, S, P)$$

$$V = \{S\}$$

variables

$$T = \{a, b\}$$

terminals

start variable

Derivation Order

- Consider the following example grammar

with 5 productions:

1. $S \rightarrow AB$
2. $A \rightarrow aaA$
3. $A \rightarrow \lambda$
4. $B \rightarrow Bb$
5. $B \rightarrow \lambda$

1. $S \to AB$ 2. $A \to aaA$ 4. $B \to Bb$

3. $A \to \lambda$ 5. $B \to \lambda$

Leftmost derivation order of string : aab

$$S \overset{1}{\Rightarrow} AB \overset{2}{\Rightarrow} aaAB \overset{3}{\Rightarrow} aaB \overset{4}{\Rightarrow} aaBb \overset{5}{\Rightarrow} aab$$

At each step, we substitute the leftmost variable

1. $S \to AB$ 2. $A \to aaA$ 4. $B \to Bb$

3. $A \to \lambda$ 5. $B \to \lambda$

Rightmost derivation order of string : aab

$$S \overset{1}{\Rightarrow} AB \overset{4}{\Rightarrow} ABb \overset{5}{\Rightarrow} Ab \overset{2}{\Rightarrow} aaAb \overset{3}{\Rightarrow} aab$$

At each step, we substitute the rightmost variable

Grammar for mathematical expressions

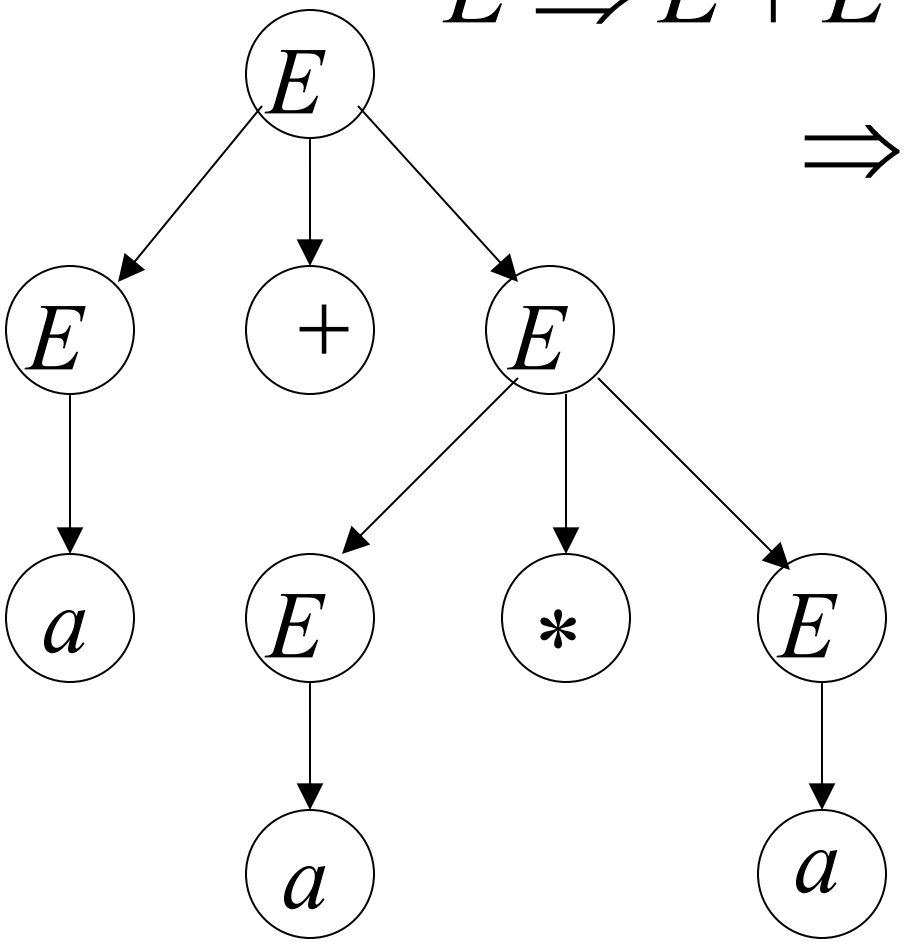
$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

Example strings:

$$(a + a) * a + (a + a * (a + a))$$

Denotes any number

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$



$$E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E$$

$$\Rightarrow a + a * E \Rightarrow a + a * a$$

A leftmost derivation for

$$a + a * a$$

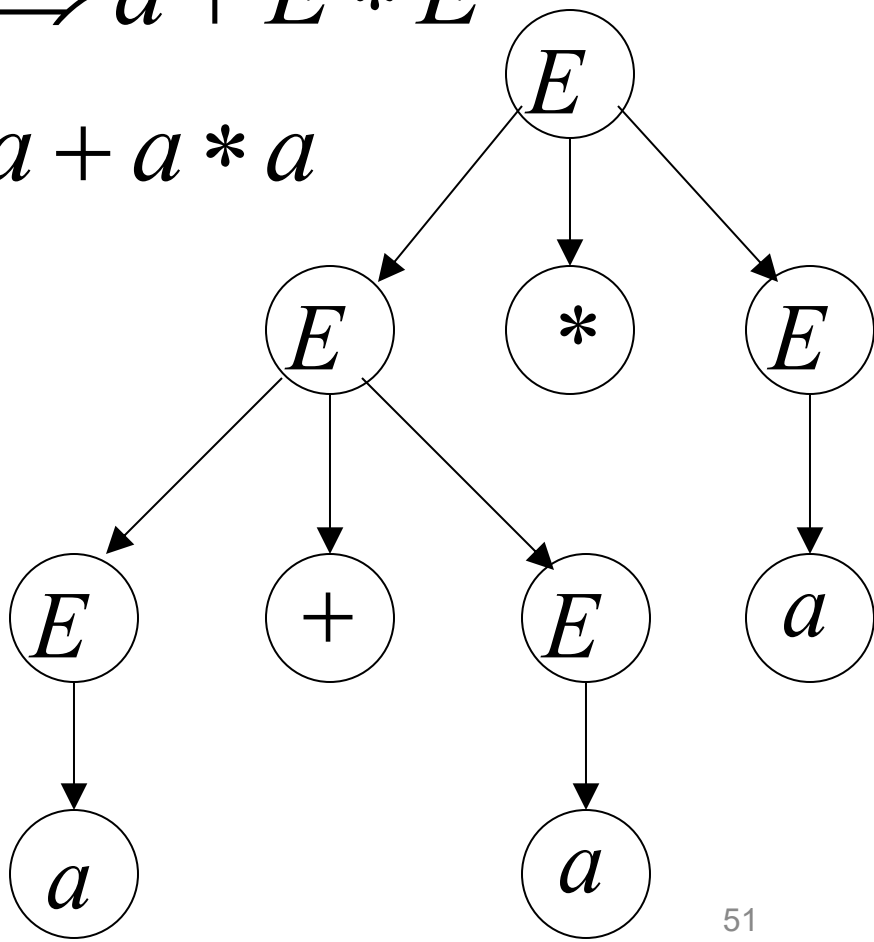
$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

$$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E$$

$$\Rightarrow a + a * E \Rightarrow a + a * a$$

Another leftmost derivation for

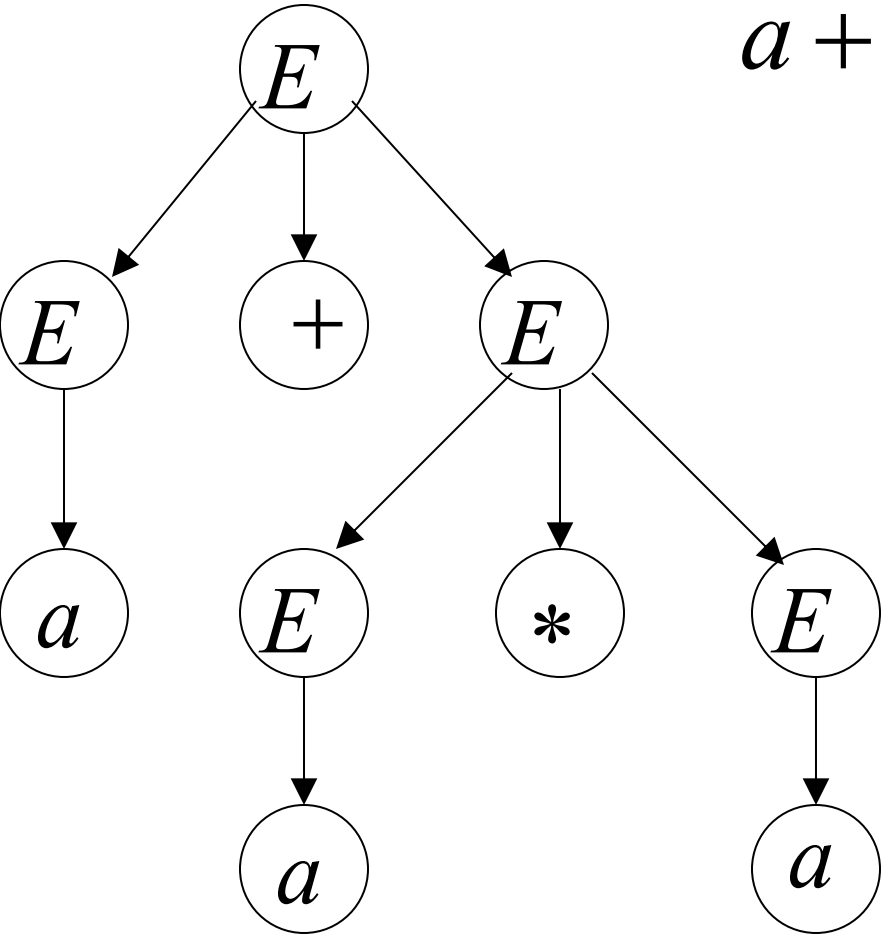
$$a + a * a$$

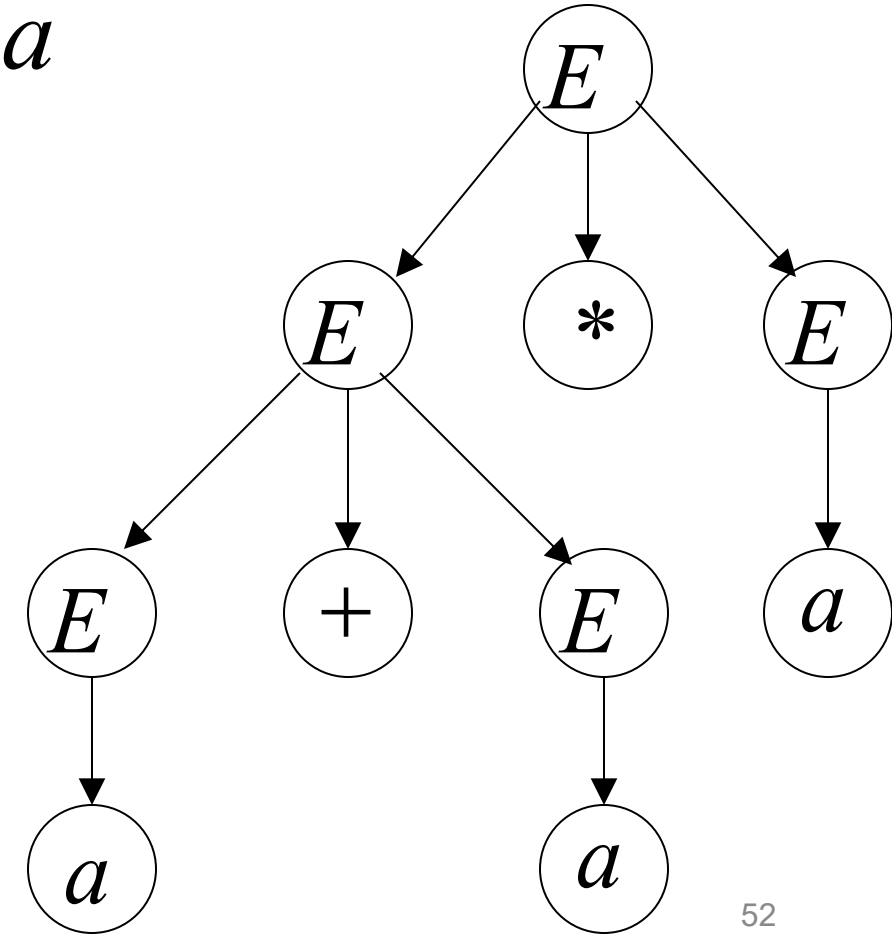


$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

Two derivation trees for

$$a + a * a$$





Two different derivation trees may cause problems in applications which use the derivation trees:

- Evaluating expressions
- In general, in compilers for programming languages

Ambiguous Grammar:

A context-free grammar **G** is ambiguous if there is a string which has: $w \in L(G)$

two different derivation trees

or

two leftmost derivations

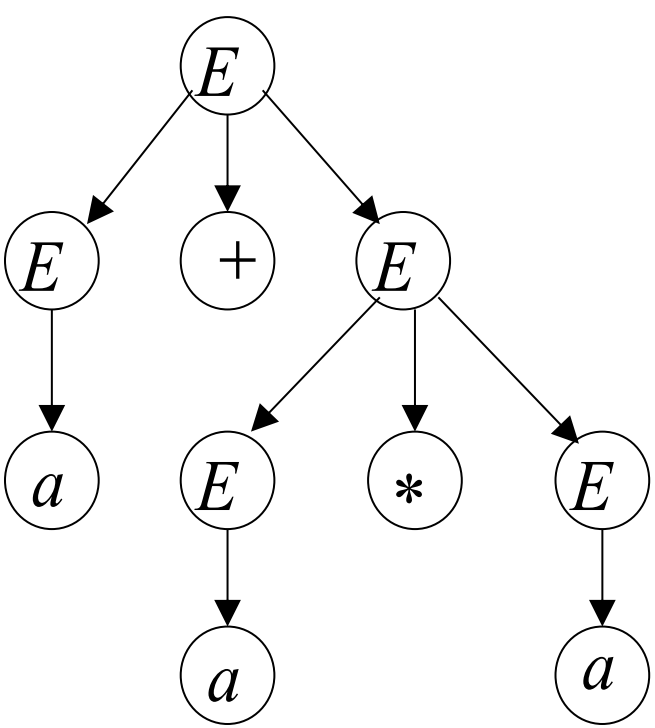
(Two different derivation trees give two different leftmost derivations and vice-versa)

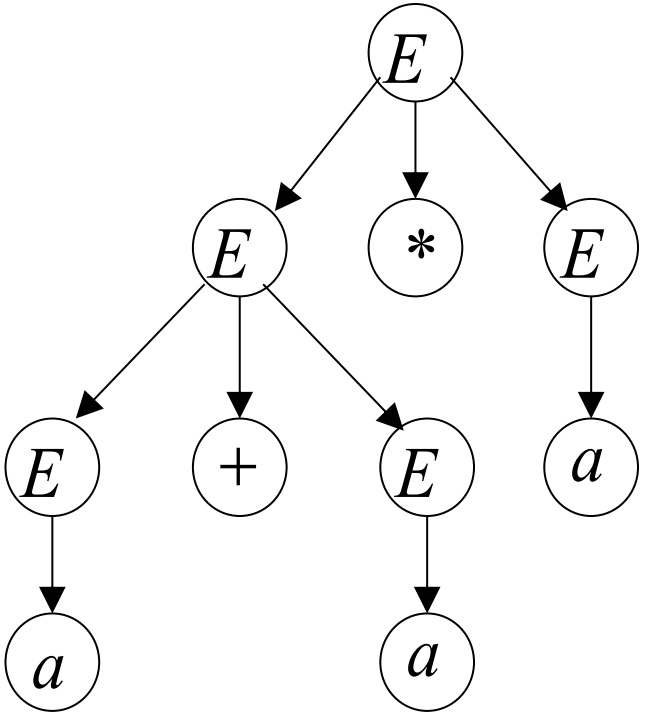
Example:

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

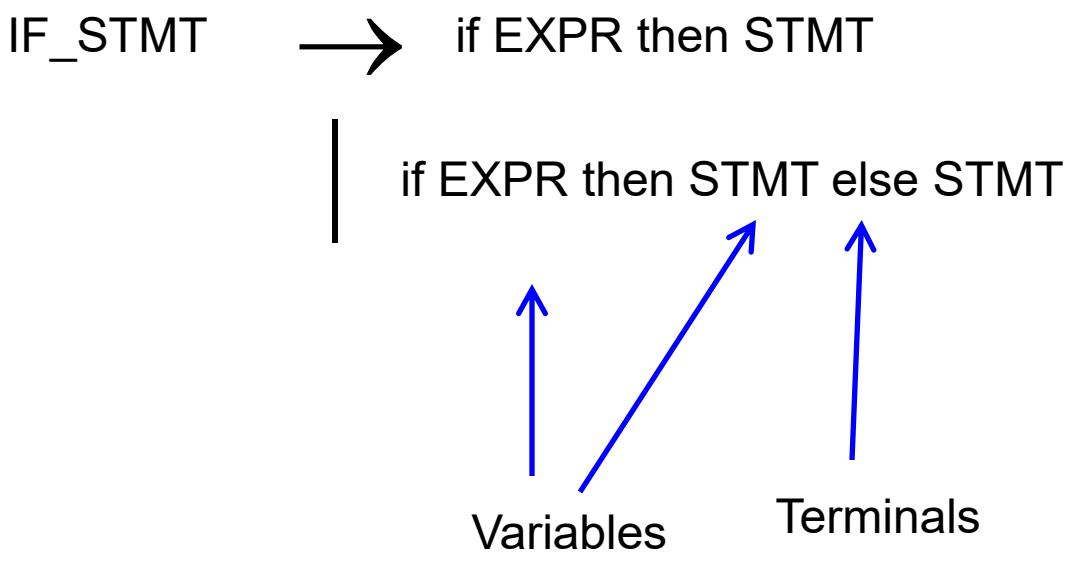
this grammar is ambiguous since

string $a + a * a$ has two derivation trees



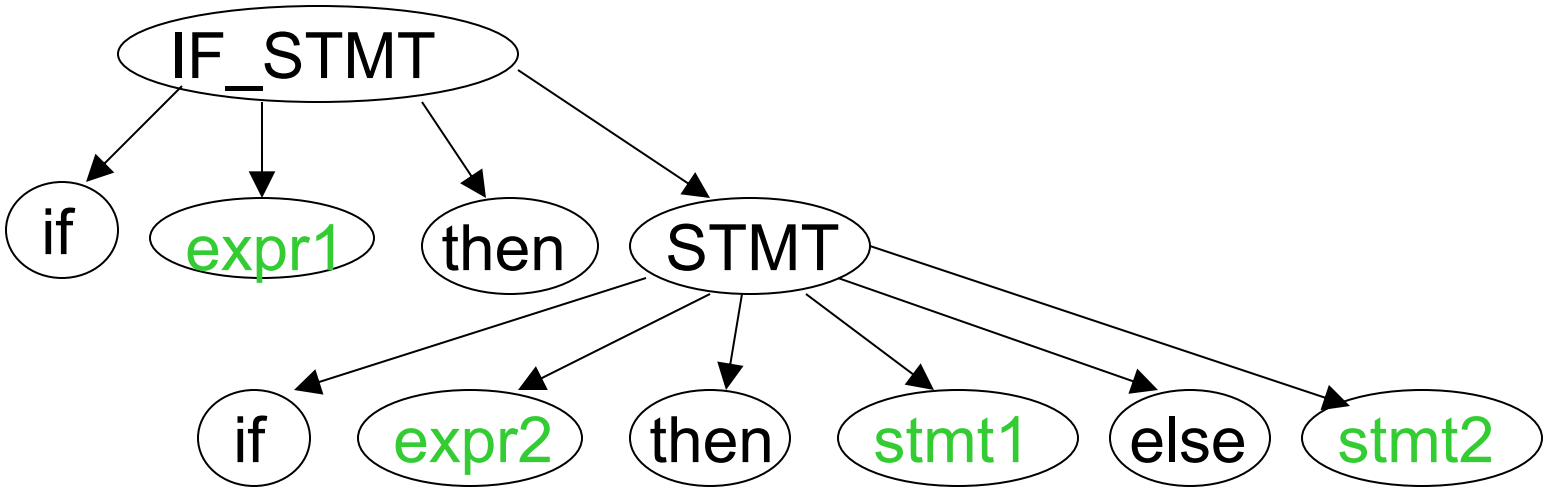


Another ambiguous grammar:

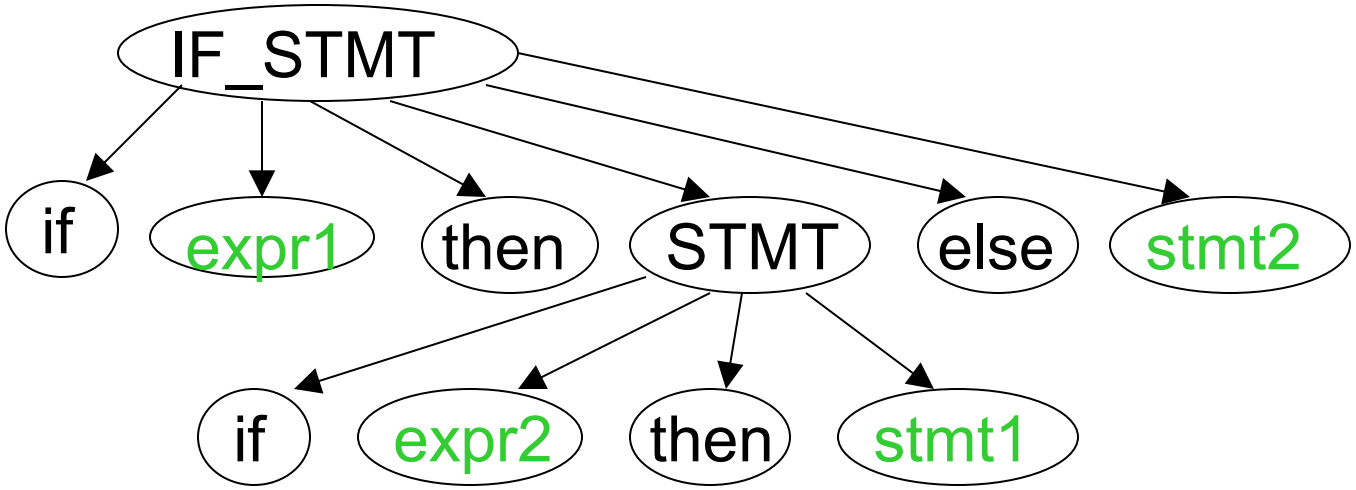


Very common piece of grammar in programming languages

If expr1 then if expr2 then stmt1 else stmt2



Two derivation trees



In general, ambiguity is bad and we want to remove it

Sometimes it is possible to find a non-ambiguous grammar for a language

But, in general we cannot do so

A successful example:

Ambiguous Grammar

$$E \to E + E$$
$$E \to E * E$$
$$E \to (E)$$
$$E \to a$$

Equivalent

Non-Ambiguous Grammar

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid a$$

generates the same language

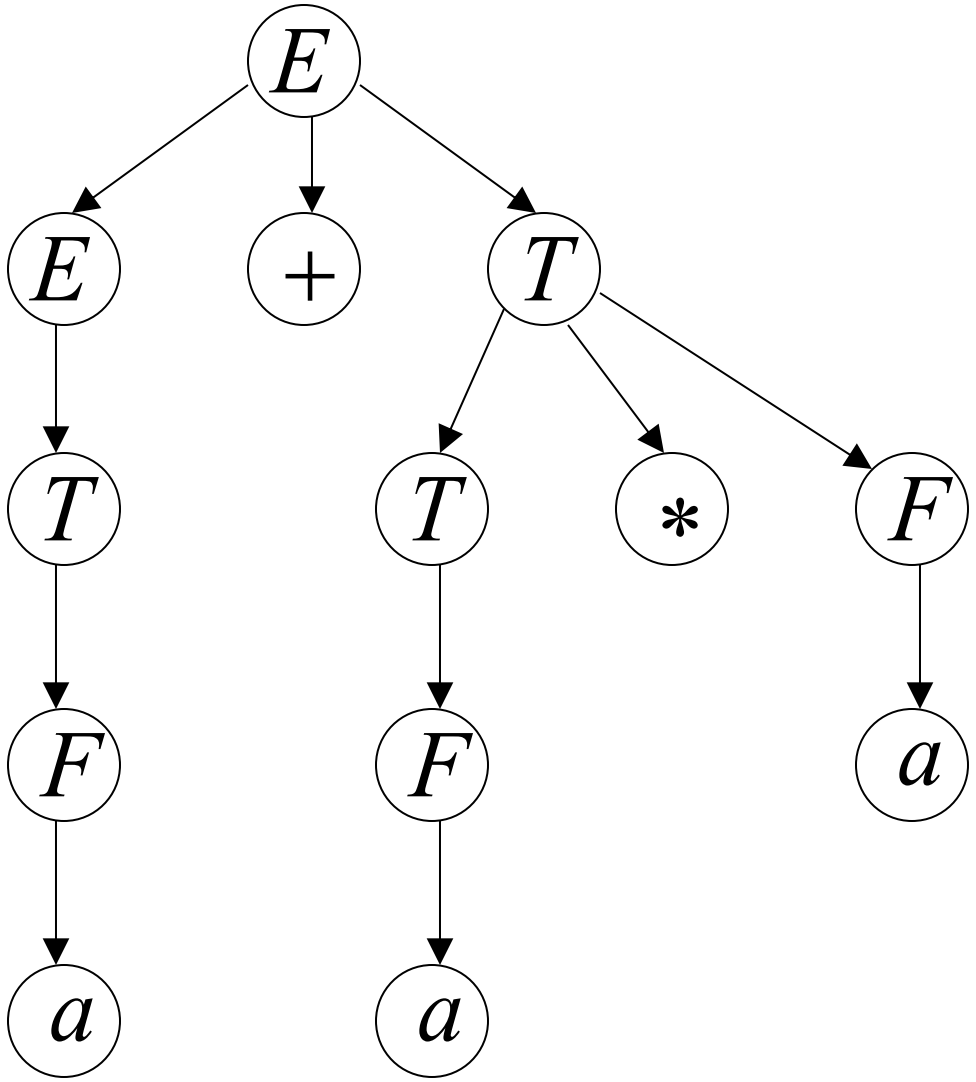
$$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T*F$$

$$\Rightarrow a+F*F \Rightarrow a+a*F \Rightarrow a+a*a$$

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow (E) \mid a$$



Unique derivation tree for

$$a+a*a$$

Elimination of ambiguity in grammars

If the grammar is in the form of

$S \rightarrow \alpha S \beta S \gamma \mid \alpha 1 \mid \alpha 2 \mid \ldots \mid \alpha n$

Is replaced by

$S \rightarrow \alpha S \beta S^1 \gamma \mid S^1$

$S^1 \rightarrow \alpha 1 \mid \alpha 2 \mid \ldots \mid \alpha n$

Ex:

E-> E+E | E*E | id | (E) is replaced by

E -> E+T | T

T->T*F | F

F -> id | (E)

Top down Parsers

- All top down parsers use leftmost derivation in order to construct parse tree.
- The parse tree is created from top to bottom (i.e from root node to leaf nodes).
- To apply any top down parsing method , the grammar(CFG) must unambiguous.

Top-Down Parsing

- Recursive-Descent Parsing(with backtracking)
  - Backtracking is needed (If a choice of a production rule does not work, we backtrack to try other alternatives.)
  - It is a general parsing technique, but not widely used.
  - Not efficient.
  - The recursive descent parsing can not handle *left recursive* grammars , thus we need to eliminate left recursion property from the grammar.
  - And also the grammar must be *unambiguous.*

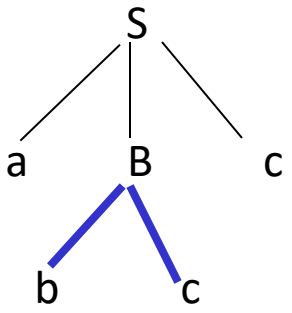
Recursive-Descent Parsing (uses Backtracking)

- Backtracking is needed.
- It tries to find the left-most derivation.

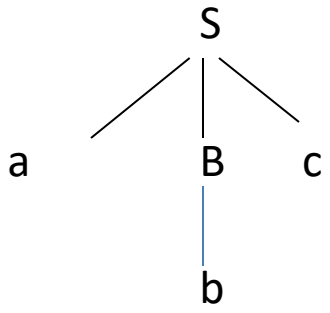
S → aBc

B → bc | b

input: **abc**



fails, backtrack



Left Recursion

- A grammar is ***left recursive*** if it has a non-terminal A such that there is a derivation.

  $\mathbf{A \Rightarrow A\alpha}$ for some string α

- Top-down parsing techniques **cannot** handle left-recursive grammars.
- So, we have to convert our left-recursive grammar into an equivalent grammar which is non left-recursive.
- The left-recursion may appear in a single step of the derivation (*immediate left-recursion*), or may appear in more than one step of the derivation.

Immediate Left-Recursion

$A \rightarrow A\,\alpha \mid \beta$ where β does not start with A

$\Downarrow$ eliminate immediate left recursion

$\mathbf{A \rightarrow \beta\, A'}$

$\mathbf{A' \rightarrow \alpha\, A' \mid \varepsilon}$ an equivalent grammar

In general,

$A \rightarrow A\,\alpha_1 \mid \ldots \mid A\,\alpha_m \mid \beta_1 \mid \ldots \mid \beta_n$ where $\beta_1 \ldots \beta_n$ do not start with A

$\Downarrow$ eliminate immediate left recursion

$A \rightarrow \beta_1\, A' \mid \ldots \mid \beta_n\, A'$

$A' \rightarrow \alpha_1\, A' \mid \ldots \mid \alpha_m\, A' \mid \varepsilon$ an equivalent grammar

Immediate Left-Recursion -- Example

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow id \mid (E)$

$\Downarrow$ eliminate immediate left recursion

$E \rightarrow T\ E'$

$E' \rightarrow +T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *F\ T' \mid \varepsilon$

$F \rightarrow id \mid (E)$

Left-Recursion -- Problem

- A grammar cannot be immediately left-recursive, but it still can be left-recursive.
- By just eliminating the immediate left-recursion, we may not get a grammar which is not left-recursive.

$S \rightarrow Aa \mid b$

$A \rightarrow Sc \mid d$ This grammar is not immediately left-recursive, but it is still left-recursive.

$\underline{S} \Rightarrow Aa \Rightarrow \underline{S}ca$ or

$\underline{A} \Rightarrow Sc \Rightarrow \underline{A}ac$ causes to a left-recursion

- So, we have to eliminate all left-recursions from our grammar

Eliminate left recursion from the following grammars

1) $S \rightarrow AA / 0$
   $A \rightarrow SS / 1$

2) $A \rightarrow Bb / Cc$
   $B \rightarrow Ab / Cb / b$
   $C \rightarrow Ac / Bc / c$

3) $S \rightarrow (L) / a$
   $L \rightarrow L,S / S$

Top-Down Parsing

- Recursive Predictive Parsing(without backtracking)
  - no backtracking
  - Efficient than recursive descent parsing
  - Implemented using **transition diagrams**.
  - Can not handle the productions with common sub expressions , thus we need to eliminate common sub expressions in the productions of grammar called ***left factoring*** the grammar
  - And also the grammar must be ***unambiguous*** and ***non left recursive***

Predictive Parser (example)

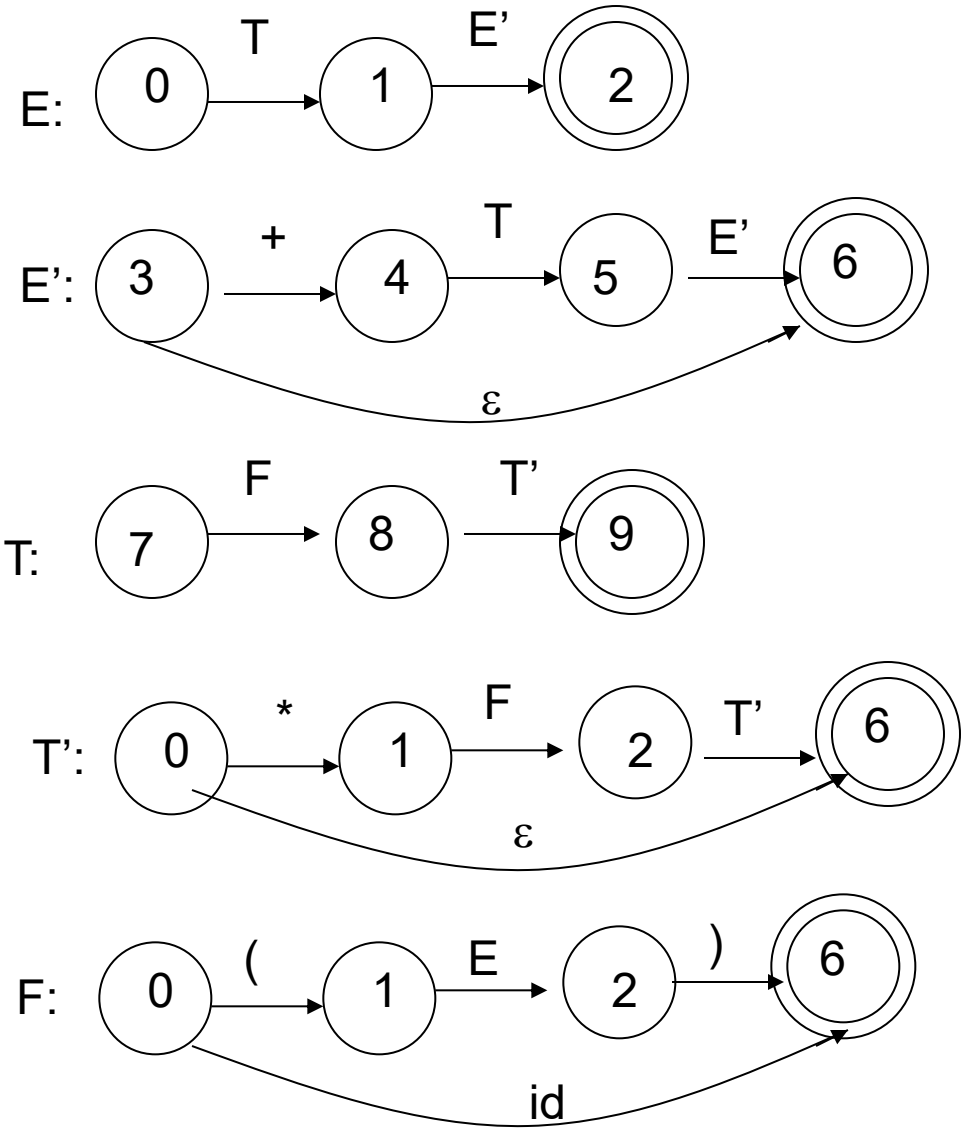
$E \rightarrow T\ E'$

$E' \rightarrow +T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *F\ T' \mid \varepsilon$

$F \rightarrow id \mid (E)$



Left-Factoring
(Eliminating common sub expressions from the productions)

- In general,

  $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ where α is non-empty and the first symbols of β_1 and β_2 (if they have one)are different.

- when processing α we cannot know whether expand

  A to $\alpha\beta_1$ or

  A to $\alpha\beta_2$

- But, if we re-write the grammar as follows

  $A \rightarrow \alpha A'$

  $A' \rightarrow \beta_1 \mid \beta_2$ **so, we can immediately expand A to $\alpha A'$**

Left-Factoring – Example

$A \rightarrow ad \mid a \mid ab \mid abc \mid b$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid b \mid bc$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid bA''$

$A'' \rightarrow \varepsilon \mid c$

Eliminate left Factoring from the following grammars

1) S → iEtS / iEtSeS / a

   E → b

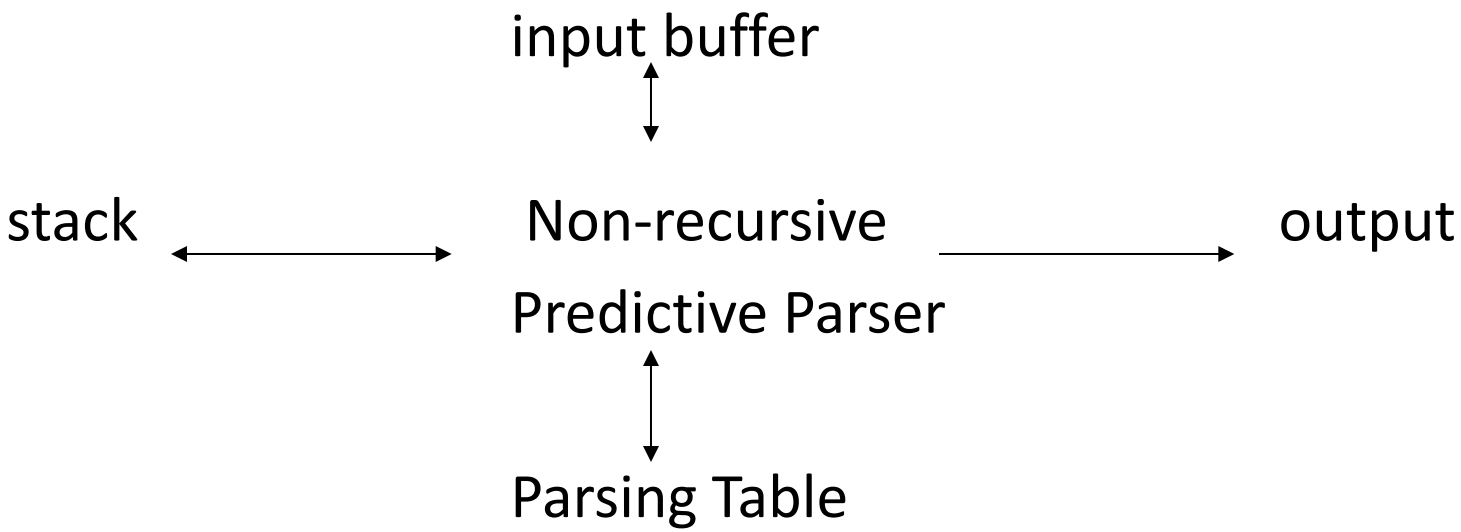
2) D → TL;

   T → int / float

   L → id,L / id

Non-Recursive Predictive Parsing -- LL(1) Parser

- Non-Recursive predictive parsing is a table-driven parser.
- It is a top-down parser.
- It is also known as LL(1) Parser.



LL(1) Parser

input buffer

- our string to be parsed. We will assume that its end is marked with a special symbol $.

output

- a production rule representing a step of the derivation sequence (left-most derivation) of the string in the input buffer.

stack

- contains the grammar symbols
- at the bottom of the stack, there is a special end marker symbol $.
- initially the stack contains only the symbol $ and the starting symbol S.
- $S ← initial stack
- when the stack is emptied (ie. only $ left in the stack), the parsing is completed.

parsing table

- a two-dimensional array M[A,a]
- each row is a non-terminal symbol
- each column is a terminal symbol or the special symbol $
- each entry holds a production rule.

LL(1) Parser – Parser Actions

- The symbol at the top of the stack (say X) and the current symbol in the input string (say a) determine the parser action.

  There are four possible parser actions:

1. **If X and a are $ ➔ parser halts (successful completion)**

2. **If X and a are the same terminal symbol (different from $)**

   ➔ parser pops X from the stack, and moves the next symbol in the input buffer.

3. **If X is a non-terminal**

   ➔ parser looks at the parsing table entry M[X,a]. If M[X,a] holds a production rule $X \rightarrow Y_1 Y_2 \ldots Y_k$, it pops X from the stack and pushes $Y_k, Y_{k-1}, \ldots, Y_1$ into the stack. The parser also outputs the production rule $X \rightarrow Y_1 Y_2 \ldots Y_k$ to represent a step of the derivation.

4. **none of the above ➔ error**
   - **all empty entries in the parsing table are errors.**
   - **If X is a terminal symbol different from a, this is also an error case.**

LL(1) Parser – Example1

S → aBa
B → bB | ε

	a	b	$
S	S → aBa		
B	B → ε	B → bB	

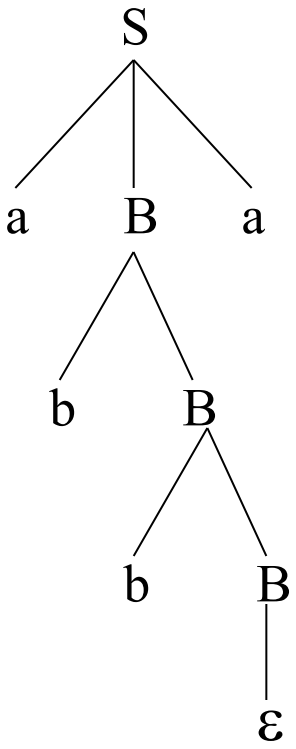
LL(1) Parsing Table

stack	**input**	**output**
$S	abba$	S → aBa
$aBa	abba$	
$aB	bba$	B → bB
$aBb	bba$	
$aB	ba$	B → bB
$aBb	ba$	
$aB	a$	B → ε
$a	a$	
$	$	accept, successful completion

LL(1) Parser – Example1 (cont.)

Outputs: $S \rightarrow aBa$ $B \rightarrow bB$ $B \rightarrow bB$ $B \rightarrow \varepsilon$

Derivation(left-most): $S \Rightarrow aBa \Rightarrow abBa \Rightarrow abbBa \Rightarrow abba$



parse tree

Constructing LL(1) Parsing Tables

- Two functions are used in the construction of LL(1) parsing tables:
  - FIRST FOLLOW

- **FIRST(α)** is a set of the terminal symbols which occur as first symbols in strings derived from α where α is any string of grammar symbols.
  if α derives to ε, then ε is also in FIRST(α) .

- **FOLLOW(A)** is the set of the terminals which occur immediately after (follow) the *non-terminal A* in the strings derived from the starting symbol.
  - terminal a is in FOLLOW(A) if $S \Rightarrow^{*} \alpha A a \beta$
  - \$ is in FOLLOW(A) if $S \Rightarrow^{*} \alpha A$

To compute FIRST (X) for all grammar symbols X, apply the following rules until no more terminals or e can be added to any FIRST set.

1. If X is terminal, then First (X) is {X}.

2. If $X \rightarrow \varepsilon$ is a production then add e to FIRST(X).

3. If X is a non terminal and $X \rightarrow Y_1 Y_k \ldots\ldots Y_k$ is a production, then place *a* in First (X) if for some i, *a* is in FIRST(Y_i) and e is in all of FIRST(Y_1), FIRST(Y_2),..., FIRST(Y_{i-1});that is, $Y_1 .. Y_{i-1} \Rightarrow^* \varepsilon$. If ε is in FIRST(Y_j) for all i = 1,2,..,k, then add ε to FIRST(X). For example, everything in FIRST(Y_1) is surely in FIRST(X). If Y_1 does not derive ε, then we add nothing more to FIRST(X), but if $Y_1 \Rightarrow^* \varepsilon$, then we add FIRST($Y_2$) and so on.

FIRST Example

1)E → TE′
2)E′ → +TE′ | ε
3)T → FT′
4)T′ → *FT′ | ε
5)F → (E) | id

1)FIRST(E) ⇒ FIRST(TE′) ⇒FIRST(FT′) ⇒ FIRST((E)) U FIRST(id) ⇒ { (,id}
FIRST(E)= { (,id}

2)FIRST(E′) = FIRST(+TE′)={+}
FIRST(ε) = {ε}
FIRST(E′)= {+, ε}

3)FIRST(T) ⇒ FIRST(FT′) ⇒ FIRST(F) = { (,id}
FIRST(T) = {(, id}

4)FIRST(T') = FIRST(*FT′) ={*}
FIRST(ε) ={ε}
FIRST(T') ={*, ε}

5)FIRST(F) ⇒ FIRST((E)) = { (} ,FIRST(id) = {id}
FIRST(F) = {(, id}

FIRST Example

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \varepsilon$
$F \rightarrow (E) \mid id$

FIRST(F) = {(,id}
FIRST(T′) = {*, ε}
FIRST(T) = {(,id}
FIRST(E′) = {+, ε}
FIRST(E) = {(,id}

FIRST(TE′) = {(,id}
FIRST(+TE′) = {+}
FIRST(ε) = {ε}
FIRST(FT′) = {(,id}
FIRST(*FT′) = {*}
FIRST(ε) = {ε}
FIRST((E)) = {(}
FIRST(id) = {id}

To compute FOLLOW (A) for all non-terminals A , apply the following rules until nothing can be added to any FOLLOW set:

1. Place $ in FOLLOW(S), where S is the start symbol and $ is the input right endmarker.

2. If there is a production A → a Bß, then everything in FIRST(ß) except for e is placed in FOLLOW(B).

3. If there is a production A → a ß, or a production A → a Bß where FIRST(ß) contains e (i.e., ß ⇒* e), then everything in FOLLOW(A) is in FOLLOW(B).

FOLLOW Example

FOLLOW(E)

(finding the occurrence of non-terminal on right side productions)

F $\rightarrow$ (E) is in the form of

$A \rightarrow \alpha B \beta$

FOLLOW(B)=FIRST(β) **(rule)**

FOLLOW(E)= FIRST( ))

FOLLOW(E)={$,)} **E is starting symbol**

2) FOLLOW(E’)

$E' \rightarrow +TE' \mid \varepsilon$

a) contain ε hence

FOLLOW(B)=(FIRST(β) - ε) U (FOLLOW(A))

$E' \rightarrow +TE'$ is in the form of $A \rightarrow \alpha B \beta$

b) FOLLOW(T)={+} U (FOLLOW(E’))

c) FOLLOW(T)={+}U ?

d) FOLLOW(E’) (occurrence of E’ on right side productions)

e) $E \rightarrow TE'$ is in the form of

f) $A \rightarrow \alpha B$

g) FOLLOW(E’)=FOLLOW(E)= {**$**, **)}**

h) FOLLOW(T)={+}U {**$**, **)} ={**+,**$**, **)}**

3) FOLLOW(T)

$T \rightarrow FT'$ is in the form of

a) $A \rightarrow \alpha B$ hence FOLLOW(B)= FOLLOW(A)

b) FOLLOW(T')=FOLLOW(T)= {+,**$**, **)}**

4) $T' \rightarrow *FT' \mid \varepsilon$

a) contain ε hence FOLLOW(B)=(FIRST(β) - ε) U (FOLLOW(A))

$T' \rightarrow *FT'$ is in the form of $A \rightarrow \alpha B \beta$

b) FOLLOW(F)={*}U(FOLLOW(T'))

c) FOLLOW(F)={*}U ?

d) FOLLOW(T') (occurrence of T' on right side productions)

e) $T \rightarrow FT'$ is in the form of

f) $A \rightarrow \alpha B$

g) FOLLOW(T')=FOLLOW(T)= {+,**$**, **)}**

h) FOLLOW(F)={*}U {+,$,)} **={*,+,$,)}**

FOLLOW Example

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid id$

FOLLOW(E) = { $,) }

FOLLOW(E′) = { $,) }

FOLLOW(T) = { +,), $ }

FOLLOW(T′) = { +,), $ }

FOLLOW(F) = {+, *,), $ }

Compute FIRST and FOLLOW of the following Grammars

1) D → TL;
   T → int / float
   L → id,L / id
2) A → BCc / gDB
   B → ϵ / bCDE
   C → DaB / Ca
   D → ϵ / dD
   E → g^f / C

3) P → AQRbe / mn / DEl
   A → ab / ϵ
   Q → q_1q_2 / ϵ
   R→ r_1r_2 / ϵ
   D → d
   E → e

Constructing LL(1) Parsing Table -- Algorithm

- for each production rule $A \rightarrow \alpha$ of a grammar G
  - for each terminal a in FIRST(α)
    ➔ add $A \rightarrow \alpha$ to M[A, a]
  - If ε in FIRST(α)
    ➔ for each terminal b in FOLLOW(A) add $A \rightarrow \alpha$ to M[A, b]
  - If ε in FIRST(α) and $ in FOLLOW(A)
    ➔ add $A \rightarrow \alpha$ to M[A,$]

- All other undefined entries of the parsing table are error entries.

Constructing LL(1) Parsing Table -- Example

$E \rightarrow TE'$	FIRST(TE')={(,id}	➔ $E \rightarrow TE'$ into M[E,(] and M[E,id]
$E' \rightarrow +TE'$	FIRST($+TE'$)={+}	➔ $E' \rightarrow +TE'$ into M[E′,+]
$E' \rightarrow \varepsilon$	FIRST(ε)={ε} but since ε in FIRST(ε) and FOLLOW(E′)={$,)}	➔ none ➔ $E' \rightarrow \varepsilon$ into M[E′,$] and M[E′,)]
$T \rightarrow FT'$	FIRST(FT')={(,id}	➔ $T \rightarrow FT'$ into M[T,(] and M[T,id]
$T' \rightarrow *FT'$	FIRST($*FT'$)={*}	➔ $T' \rightarrow *FT'$ into M[T′,*]
$T' \rightarrow \varepsilon$	FIRST(ε)={ε} but since ε in FIRST(ε) and FOLLOW(T′)={$,),+}	➔ none ➔ $T' \rightarrow \varepsilon$ into M[T′,$], M[T′,)] and M[T′,+]
$F \rightarrow (E)$	FIRST((E))={(}	➔ $F \rightarrow (E)$ into M[F,(]
$F \rightarrow id$	FIRST(id)={id}	➔ $F \rightarrow id$ into M[F,id]

LL(1) Parser – Example2

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \varepsilon$
$F \rightarrow (E) \mid id$

F->id

FIRST(id) = { id }

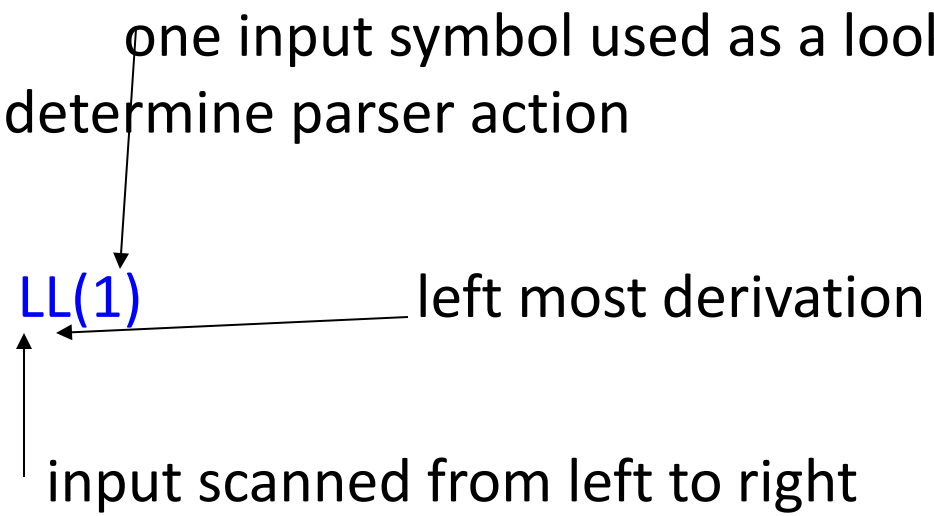
	id	**+**	*****	**(**	**)**	**$**
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

LL(1) Parser – Example2

stack	input	output
$E	id+id$	$E \rightarrow TE'$
$E'T	id+id$	$T \rightarrow FT'$
$E' T'F	id+id$	$F \rightarrow id$
$ E' T'id	id+id$	
$ E' T'	+id$	$T' \rightarrow \varepsilon$
$ E'	+id$	$E' \rightarrow +TE'$
$ E' T+	+id$	
$ E' T	id$	$T \rightarrow FT'$
$ E' T' F	id$	$F \rightarrow id$
$ E' T'id	id$	
$ E' T'	$	$T' \rightarrow \varepsilon$
$ E'	$	$E' \rightarrow \varepsilon$
$	$	accept

LL(1) Grammars

- A grammar whose parsing table has no multiply-defined entries is said to be LL(1) grammar.



- The parsing table of a grammar may contain more than one production rule. In this case, we say that it is not a LL(1) grammar.

A Grammar which is not LL(1)

$S \rightarrow i\ C\ t\ S\ E \mid a$
$E \rightarrow e\ S \mid \varepsilon$
$C \rightarrow b$

FOLLOW(S) = { $,e }
FOLLOW(E) = { $,e }
FOLLOW(C) = { t }

FIRST(iCtSE) = {i}
FIRST(a) = {a}
FIRST(eS) = {e}
FIRST(ε) = {ε}
FIRST(b) = {b}

	a	b	e	i	t	$
S	$S \rightarrow a$			$S \rightarrow$ iCtSE		
E			$E \rightarrow e\ S$ $E \rightarrow \varepsilon$			$E \rightarrow \varepsilon$
C		$C \rightarrow b$				

two production rules for M[E,e]

Problem ➔ ambiguity

Construct LL(1) Parsing Table for the following gramma

1) $S \rightarrow iEtSS' / a$

$S' \rightarrow eS / \epsilon$

$E \rightarrow b$

2) $E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid id$

3) $S \rightarrow 1AB / \epsilon$

$A \rightarrow 1AC / 0C$

$B \rightarrow 0S$

$C \rightarrow 1$

4) $S \rightarrow (L) / a$

$L \rightarrow SL`$

$L` \rightarrow ,S\ L` / \epsilon$